

TurboSheet (Rashiq) PRD

v2.0 Update

High-Concurrency Rust Browser Automation for the AI Era

Version 2.0 | Status: Active Draft | May 2026

1. Executive Summary

TurboSheet (codename *Rashiq*) is a high-performance browser automation framework written in Rust. While the initial v1.0 focused on token-efficient AI state extraction, the v2.0 pivot addresses the mid-2026 landscape where competitors (Playwright/Puppeteer) have adopted native AI features and WebDriver BiDi. TurboSheet v2.0 strictly targets **infrastructure cost reduction, extreme concurrency, and raw execution speed** by leveraging lightweight Rust threads (Tokio) and zero-cost FFI via `napi-rs` to bypass Node.js limitations.

2. Updated Problem Statement

- **The Agentic Swarm Bottleneck:** Running 50+ parallel AI agents in Node.js using Playwright heavily taxes CPU/RAM due to the process-per-context architecture. Infrastructure costs for scaling Web Agents are currently prohibitive.
- **BiDi Stream Overload:** While Puppeteer and Playwright now support WebDriver BiDi, streaming massive amounts of bidirectional browser events directly into the single-threaded Node.js event loop causes severe latency.
- **Serialization Tax:** The overhead of moving JSON-RPC data between Node.js and the browser remains a core physical limitation of existing JavaScript-first tools.

3. Product Objectives (v2.0 Pivot)

- **Extreme Concurrency:** Support 10x more parallel browser contexts per server compared to Playwright.
- **Cost Efficiency:** Reduce server RAM/CPU consumption by 60% for large-scale scraping and agent swarms.

- **Edge-Ready:** Introduce WebAssembly (WASM) portability for running lightweight automation logic close to the edge.

4. Core Differentiating Features

4.1 Native BiDi Stream Pre-Processor

Instead of passing raw WebDriver BiDi streams to Node.js, TurboSheet ingests, filters, and processes the immense event stream natively in Rust. Only the finalized, condensed state is sent across the FFI bridge, keeping the Node.js event loop completely clear.

4.2 Swarm Threading Architecture

Utilizing Rust's Tokio runtime, browser contexts are managed as lightweight threads rather than heavy Node.js child processes. This unlocks "Swarm Mode," enabling massive horizontal scaling on a single machine.

4.3 Dual-Core Engine (Retained & Enhanced)

Mode	Core Driver	Primary Use Case
Turbo (CDP)	Chromiumoxide	High-throughput Data Scraping, legacy Enterprise SaaS testing.
Stealth (BiDi)	Rustenium	Modern AI Agents, multi-touch emulation, complex event-driven workflows.

5. Technical Architecture Update

The Stack (Mid-2026):

- **Core Engine:** Rust (Tokio for async, Serde for zero-copy FFI).
- **Bridge:** napi-rs (Node.js) + Experimental WASM compilation targets.
- **Event Handling:** Rust-native BiDi event filter (filters noise before hitting JS).
- **Wait Engine:** Native Selector Resolution (polling happens in Rust).

6. Updated Roadmap

Phase 1: Swarm MVP	NAPI-RS bridge + Rust-native Selector Engine + Basic Tokio threading.
Phase 2: BiDi Shield	Rustenium integration with the native BiDi stream pre-processor.
Phase 3: Edge Portability	WASM compilation for running automation logic in sandboxed edge environments.
Phase 4: Ecosystem	Drop-in replacement wrappers for existing Playwright configurations.